

# 数字图像处理实验报告

## 基于 YOLOv5 的有害垃圾智能检测系统

面向混合垃圾场景的有害垃圾快速筛查

项目成员：刘晋嶂、邓荣基、袁致朗

模型训练：刘晋嶂

图像处理：邓荣基

技术调研与报告：袁致朗

完成日期：2026 年 6 月

基于数字图像处理与深度学习目标检测技术

### 摘要

在生活垃圾收集、转运和分拣过程中，干电池、过期药品、药膏及其他具有毒性、腐蚀性或潜在环境危害的物品，可能被混入大量普通垃圾。若这些有害垃圾未在前端及时识别并分离，它们可能进入压缩、破碎、焚烧、填埋或资源化利用等后续环节，造成有害物质泄漏、设备污染和二次环境风险。与此同时，环卫工人面对的通常不是背景干净、摆放规则的单件垃圾，而是数量庞大、相互遮挡且外观复杂的混合垃圾，单纯依靠人工逐件检查效率低、劳动强度高，并可能增加直接接触风险。

本项目围绕上述现实问题，设计并实现了一套基于 YOLOv5s 的垃圾目标检测系统。系统首先使用数字图像处理流水线对上传图像进行尺寸归一化、Gaussian Blur 去噪、CLAHE 局部对比度增强及 Unsharp Masking 锐化，再由 YOLOv5s 对图像中的垃圾进行分类与定位。当检测结果中出现干电池、过期药品、药膏、充电宝和化妆品瓶等有害垃圾时，Web 前端会进行醒目提示。实验采用开源且已完成目标框标注的垃圾图像数据集进行迁移学习，最终模型在验证集上获得 Precision 0.5476、Recall 0.5506、mAP@0.5 0.5612 和 mAP@0.5:0.95 0.3408。结果表明，本项目实现了从图像预处理、模型训练到前端部署的完整流程，并初步验证了利用视觉检测技术辅助环卫人员快速筛查有害垃圾的可行性。

**关键词：**有害垃圾；目标检测；YOLOv5；图像预处理；垃圾分类；环境保护

目录

|       |                 |    |
|-------|-----------------|----|
| 1     | 项目介绍            | 3  |
| 1.1   | 项目背景            | 3  |
| 1.2   | 项目目标与现实价值       | 3  |
| 1.3   | 系统功能与技术路线       | 3  |
| 2     | 数据采集与处理的详细过程    | 4  |
| 2.1   | 早期数据采集与手工标注尝试   | 4  |
| 2.2   | 采用开源标注数据集       | 4  |
| 2.3   | 图像预处理流程         | 5  |
| 2.3.1 | 预处理实验对象与批量处理设计  | 5  |
| 2.3.2 | 处理组件与现实作用       | 6  |
| 2.3.3 | 关键算法原理          | 7  |
| 2.3.4 | 多类别预处理效果展示      | 8  |
| 2.4   | 数据划分与标注格式       | 9  |
| 3     | YOLOv5s 模型与训练参数 | 9  |
| 3.1   | 选择 YOLOv5s 的原因  | 9  |
| 3.2   | YOLOv5s 训练参数设置  | 9  |
| 3.3   | 评价指标            | 10 |
| 4     | 检测效果展示          | 11 |
| 4.1   | 整体训练结果          | 11 |
| 4.2   | 验证集检测效果         | 12 |
| 4.3   | 有害垃圾检测与前端系统     | 13 |
| 5     | 相关讨论            | 15 |
| 5.1   | 当前系统的有效性        | 15 |
| 5.2   | 局限性分析           | 15 |
| 5.3   | YOLOv6 与未来模型升级  | 15 |
| 6     | 总结与未来工作         | 16 |

|     |                |    |
|-----|----------------|----|
| 7   | 团队成员贡献         | 17 |
| A   | 主要程序与使用方式      | 17 |
| A.1 | 程序结构 . . . . . | 17 |
| A.2 | 运行方式 . . . . . | 18 |

# 1 项目介绍

## 1.1 项目背景

生活垃圾处理并不是单一的“投放后消失”过程，而是一条包含收集、运输、暂存、分拣、压缩、破碎、焚烧、填埋和资源化利用等环节的处理链条。若干电池、过期药品等有害垃圾在源头被混入普通垃圾，污染风险会沿着处理链向下游传递。例如，电池破损后可能释放电解液和重金属；药品包装或残留药物可能进入渗滤液；部分含化学物质的容器在压缩、破碎和焚烧过程中还可能带来额外风险。

在现实工作中，环卫工人需要从大量混合垃圾中发现这些目标。人工检查具有直观性，但存在三个明显限制：第一，垃圾数量大、更新速度快，逐件检查的时间成本高；第二，垃圾堆中目标常被遮挡、污染或压扁，容易漏检；第三，直接翻找可能使工作人员接触尖锐、腐蚀性或存在卫生风险的物品。因此，能够在图像中自动定位可疑有害垃圾的视觉系统，可以作为人工分拣前的一道辅助筛查工具。

## 1.2 项目目标与现实价值

本项目的核心目标不是取代环卫工人，而是为其提供一种快速、低接触的辅助判断方式。工作人员可以对垃圾堆或待分拣区域拍照，系统在短时间内给出垃圾类别、位置和置信度，并对有害垃圾进行重点提醒。其现实价值主要体现在以下方面：

1. **提高筛查效率。**系统能够同时检查图像中的多个目标，帮助工作人员优先关注疑似有害垃圾区域，减少盲目翻找。
2. **降低职业接触风险。**先通过图像完成初步筛查，可以减少工作人员直接接触未知垃圾的频率。
3. **阻断污染向下游传播。**尽早将有害垃圾从普通垃圾流中分离，有助于降低其进入压缩、破碎、焚烧和填埋环节后造成污染的可能性。
4. **形成可扩展的技术基础。**当前系统可部署为本地 Web 应用，未来可进一步接入固定摄像头、移动终端或垃圾分拣设备。

## 1.3 系统功能与技术路线

系统采用“图像预处理—目标检测—风险判定—前端展示”的技术路线。用户上传图片后，系统首先增强图像质量，再使用 YOLOv5s 识别并定位垃圾。当检测类别属于预先定义的有害垃圾集合时，前端以红色警告框进行重点提醒。

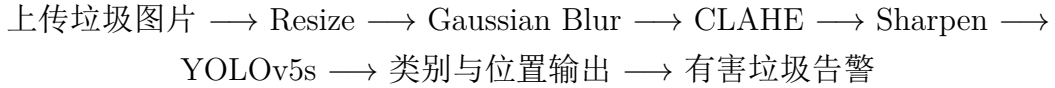


图 1 系统整体处理流程

## 2 数据采集与处理的详细过程

### 2.1 早期数据采集与手工标注尝试

项目初期，团队尝试完全自行构建垃圾检测数据集。图片主要来自团队成员自行拍摄，并使用生成式 AI 辅助生成部分垃圾图片，以补充不同类别和外观。自行拍摄能够获得与实际场景较接近的图像，AI 生成也能够一定程度上扩充样本，但受时间、可获取垃圾类别、拍摄条件和生成质量限制，最终能够使用的数据规模仍然有限。

为了将这些图片用于目标检测训练，团队使用 LabelImg 对图片进行手工标注，逐张绘制垃圾目标框并填写类别标签，共完成约 150 张图片的标注。该过程使团队实际掌握了目标检测数据标注、类别定义、目标框质量检查和 YOLO 标签格式转换等完整流程，也直观认识到高质量数据集构建需要投入大量人工成本。

然而，使用约 150 张手工标注图片进行早期训练后，模型检测效果仍不理想。首先，150 张图片难以覆盖多种垃圾类别及其外观差异；其次，部分目标在背景、光照和拍摄角度方面较为相似，数据多样性不足；最后，真实垃圾堆中常见的遮挡、污损、压扁、小目标和密集堆叠状态在早期数据中覆盖不足。目标检测模型不仅需要认识物体的典型外观，还需要学习不同尺度、角度、光照、遮挡和背景条件下的变化。有限且分布单一的数据容易导致模型过拟合，即模型可能记住训练图片，却无法稳定识别新的垃圾照片。

因此，转向规模更大、类别更丰富并且已经完成规范标注的开源数据集，并不是简单地替代原有工作，而是在早期实验结果基础上作出的必要技术选择。团队自行采集和使用 LabelImg 标注的经历，为后续检查开源数据集标注质量、理解数据配置和分析模型问题提供了重要基础。

### 2.2 采用开源标注数据集

为解决样本数量与标注质量不足的问题，团队转而调研网络上的开源垃圾图片数据集，最终采用 GitCode 平台上的开源垃圾数据集 `garbage_datasets`[3]。该数据集包含多种生活垃圾图像，并已提供目标框和类别标签，可按 YOLO 标准格式组织为 `images` 与 `labels` 目录，其中训练集和验证集分别放置在 `train` 与 `val` 子目录。

数据集地址为：[https://ai.gitcode.com/ai53\\_19/garbage\\_datasets.git](https://ai.gitcode.com/ai53_19/garbage_datasets.git)。

采用现有标注数据集具有明确的工程价值：一方面，大规模样本为模型学习真实外观差异提供基础；另一方面，已有标注可以显著减少重复劳动，使团队能够将更多精力投入图像处理、训练参数调整和应用部署。当前模型共包含 40 个垃圾类别，其中与有害垃圾筛查直接相关的类别包括 DryBattery、ExpiredDrugs、Ointment、Powerbank 和 CosmeticBottles。

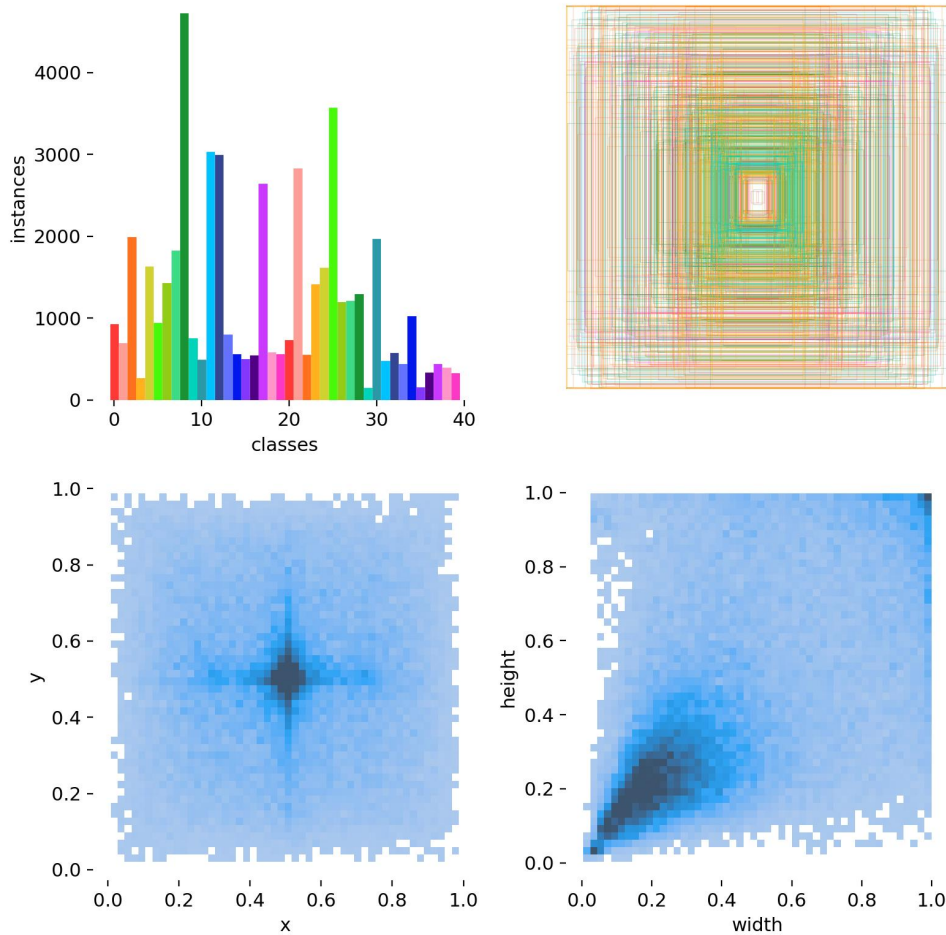


图 2 数据集目标框与类别分布可视化

## 2.3 图像预处理流程

为了改善拍摄图片中尺寸过大、局部对比度不足、噪声和边缘模糊等问题，项目在根目录的 `preprocess.py` 中实现了完整预处理流程，并将其接入 Web 检测 Pipeline。预处理不仅服务于模型输入，还对应真实使用场景中的具体问题。

### 2.3.1 预处理实验对象与批量处理设计

除在线检测 Pipeline 外，团队还在 `preprocess/` 文件夹中建立了独立的预处理实验模块，用于系统验证不同类型图片经过处理后的视觉变化。该模块批量处理了两组

图片：一组为 50 张数字图像处理课程实验图片，用于检查算法对一般图像的适用性；另一组为 48 张按类别整理的有害垃圾照片，用于重点观察电池、药品、荧光灯管和农药瓶等风险目标的纹理、标签与边缘变化。

批量处理程序会递归读取各类别目录，将处理结果保存至对应输出目录，并为每个类别自动生成原图与处理后图像的对比图。这一设计使预处理不再只是针对单张图片的演示，而成为可重复执行、可按类别检查结果的数据处理工具。考虑到 Windows 环境中数据目录和图片名称可能包含中文，程序还使用 `np.frombuffer + cv2.imdecode` 和 `cv2.imencode + tofile` 实现中文路径兼容，避免 `cv2.imread` 或 `cv2.imwrite` 因路径字符问题读取失败。

Listing 1 中文路径兼容与批量目录处理

```
def imread_unicode(path):
    with open(path, "rb") as f:
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return cv2.imdecode(data, cv2.IMREAD_COLOR)

def imwrite_unicode(path, img):
    ext = os.path.splitext(path)[1]
    _, buf = cv2.imencode(ext, img)
    buf.tofile(path)
```

2.3.2 处理组件与现实作用

| 处理组件            | 参数设置                                     | 现实作用与设计目的                                  |
|-----------------|------------------------------------------|--------------------------------------------|
| Resize          | 长边最大为 1280，保持宽高比，使用 INTER_AREA           | 控制高分辨率手机照片的计算量和内存占用，同时避免非等比例缩放造成垃圾外形失真。    |
| Gaussian Blur   | 3 × 3 高斯核                                | 减弱拍摄噪声、压缩噪声和细碎纹理干扰，使模型更关注稳定的形状和区域特征。       |
| CLAHE           | clipLimit=2.0，网格大小 8 × 8，在 LAB 空间 L 通道处理 | 增强阴影、光照不均和局部低对比度区域，使垃圾堆中较暗或与背景接近的目标更容易被观察。 |
| Unsharp Masking | 模糊核 5 × 5，强度 1.5                         | 强化目标边缘与结构信息，帮助模型区分轮廓相近或边界模糊的垃圾目标。          |



### 2.3.3 关键算法原理

**Gaussian Blur 去噪** 高斯滤波使用二维正态分布为邻域像素分配权重：

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right). \quad (1)$$

本项目选择  $3 \times 3$  小尺寸高斯核，在抑制传感器噪声与 JPEG 压缩伪影的同时，尽量保留垃圾表面的文字、纹理和轮廓。该步骤被放在 CLAHE 之前，是为了防止局部对比度增强将噪声同步放大。

**CLAHE 局部对比度增强** CLAHE (Contrast Limited Adaptive Histogram Equalization) 首先将 BGR 图像转换至 LAB 颜色空间，再将图像划分为  $8 \times 8$  个局部区域，对亮度通道 L 独立执行限制对比度的直方图均衡化。裁剪阈值 `clipLimit=2.0` 用于抑制过度增强。仅处理 L 通道可以在提升暗部可见度的同时，尽量不改变 A、B 通道承载的颜色信息，从而降低色相和饱和度失真。对于垃圾堆中处于阴影、被遮挡或颜色接近背景的目标，该方法能够使边界和表面标签更加明显。

**Unsharp Masking 锐化** 反锐化掩模首先计算图像的模糊版本，再利用原图与模糊图之间的高频差异增强边缘。本项目的计算形式为：

$$I_{\text{out}} = 1.5I_{\text{src}} - 0.5I_{\text{blur}}. \quad (2)$$

其中模糊图使用  $5 \times 5$  高斯核和  $\sigma = 1.0$  生成。中等强度锐化用于恢复去噪造成的轻微软化，并突出电池文字、药品标签、灯管轮廓等判别信息，同时避免过强锐化引入明显振铃伪影。



图 3 图像预处理前后对比

核心处理代码如下：

Listing 2 图像预处理核心流程

```
def preprocess(img):  
    img = resize_image(img, max_size=1280)  
    img = denoise_gaussian(img, kernel=3)  
    img = enhance_clahe(img, clip_limit=2.0, tile_size=8)  
    img = sharpen_unsharp(img, strength=1.5)  
    return img
```

### 2.3.4 多类别预处理效果展示

独立预处理实验进一步生成了多类有害垃圾的原图与处理后对比。图 4 展示了四类代表性结果。处理后，原图暗部和低对比度区域中的纹理更加明显，部分标签文字与目标边缘更清晰；同时，去噪步骤减少了无意义的随机纹理干扰。



图 4 不同有害垃圾类别的预处理前后对比

从批量实验可以总结出：Resize 统一了处理尺度并降低计算负担；Gaussian Blur 使画面更干净，并抑制后续增强对噪声的放大；CLAHE 对暗部和局部低对比度区域改善最明显；Unsharp Masking 则恢复了经过平滑后的边缘清晰度。四个阶段按顺序衔接，分别承担标准化、去噪、增强和细节恢复的作用。

## 2.4 数据划分与标注格式

数据集使用训练集学习模型参数，使用验证集观察泛化性能并选择最佳权重。每张图片对应一个文本标签文件，每行记录类别编号及归一化后的目标框中心坐标、宽度和高度。该组织方式能够直接被 YOLOv5 的训练脚本读取。需要指出的是，验证集必须与训练集保持独立，否则验证指标会高估模型面对新图片时的能力。

## 3 YOLOv5s 模型与训练参数

### 3.1 选择 YOLOv5s 的原因

YOLO 系列将目标分类与位置回归整合到一次前向推理中，适合需要快速反馈的目标检测场景。YOLOv5s 属于较轻量的模型版本，在计算资源有限时能够兼顾检测速度与精度，也便于在普通电脑上完成推理和 Web 部署。因此，本实验使用 YOLOv5s 作为基线模型，并基于预训练权重进行迁移学习。

### 3.2 YOLOv5s 训练参数设置

本实验采用 YOLOv5s 作为目标检测模型，并基于预训练权重 `yolov5s.pt` 进行迁移学习。训练数据集采用 YOLO 标准格式进行组织，图像文件与标签文件分别存放在 `images` 和 `labels` 文件夹中，训练集和验证集分别对应 `train` 和 `val` 子目录。

本实验使用的数据集配置文件为 `garbage_datasets/data.yaml`，其中包含训练集路径、验证集路径、类别数量以及类别名称等信息。模型训练时的主要参数设置如表 2 所示。

表 2 YOLOv5s 关键训练参数

| 参数           | 设置                         |
|--------------|----------------------------|
| 基础模型         | YOLOv5s                    |
| 预训练权重        | yolov5s.pt                 |
| 输入图像尺寸       | 640 × 640                  |
| 训练轮数         | 100 epochs                 |
| Batch size   | 2                          |
| 优化器          | SGD                        |
| 初始学习率        | 0.01                       |
| Momentum     | 0.937                      |
| Weight decay | 0.0005                     |
| 数据集配置        | garbage_datasets/data.yaml |
| 训练设备         | NVIDIA 4060 GPU, CUDA 加速   |
| 结果保存路径       | runs/train/exp12           |

训练命令如下：

Listing 3 YOLOv5s 训练命令

```
python train.py --img 640 --batch 2 --epochs 100 \
--data garbage_datasets\data.yaml --weights yolov5s.pt --workers 4
```

在训练过程中，如果训练被中断，可以使用以下命令从上一次保存的权重继续训练：

Listing 4 断点续训命令

```
python train.py --resume runs/train/exp12/weights/last.pt
```

训练完成后，模型权重文件保存在 runs/train/exp12/weights 目录下。其中，best.pt 表示在验证集上表现最好的模型权重，last.pt 表示最后一次训练保存的模型权重。后续进行图片检测和 Web 部署时选择 best.pt。

### 3.3 评价指标

本实验使用 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95 评价模型性能。Precision 表示模型报告出的目标中有多少是正确的；Recall 表示真实目标中有多少被模型找到。对于有害垃圾筛查任务，Recall 尤其重要，因为漏检意味着有害垃圾可能继续进入下游处理环节。

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

## 4 检测效果展示

### 4.1 整体训练结果

模型训练至第 100 个 epoch 后，验证集整体指标如表 3 所示。当前模型已具备一定识别能力，但整体 Precision 与 Recall 均约为 55%，说明模型仍存在较明显误检与漏检，适合作为原型系统和辅助筛查工具，尚不足以替代人工判断。

表 3 模型最终验证指标

| Epoch | Precision | Recall | mAP@0.5 | mAP@0.5:0.95 |
|-------|-----------|--------|---------|--------------|
| 99    | 0.5476    | 0.5506 | 0.5612  | 0.3408       |

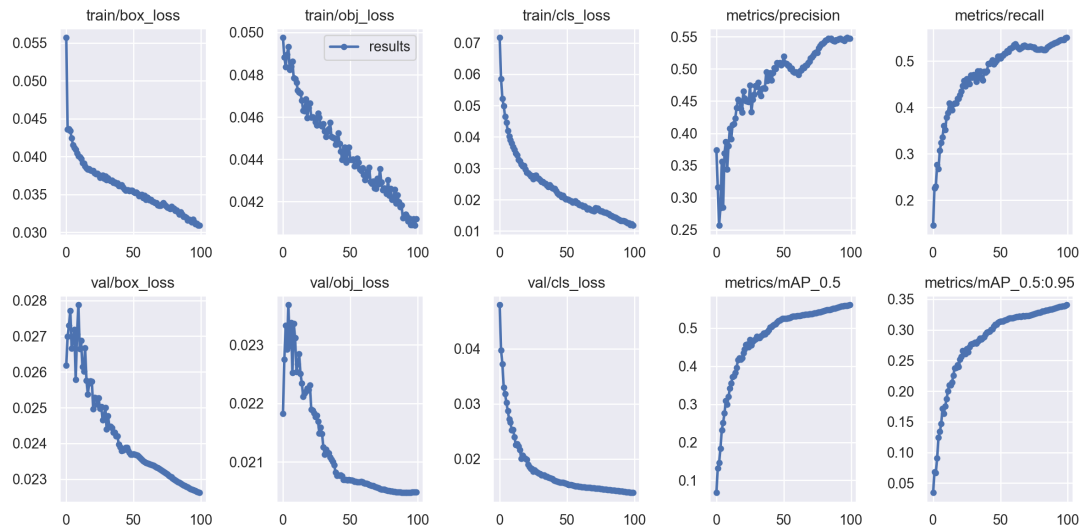


图 5 训练与验证指标变化曲线

F1-Confidence 曲线显示，全类别综合 F1 的较优置信度阈值约为 0.135。实际系统默认阈值设置为 0.15，并允许用户调整。对于有害垃圾筛查，可以适当降低阈值以提高 Recall，但同时必须接受误报增加的代价。



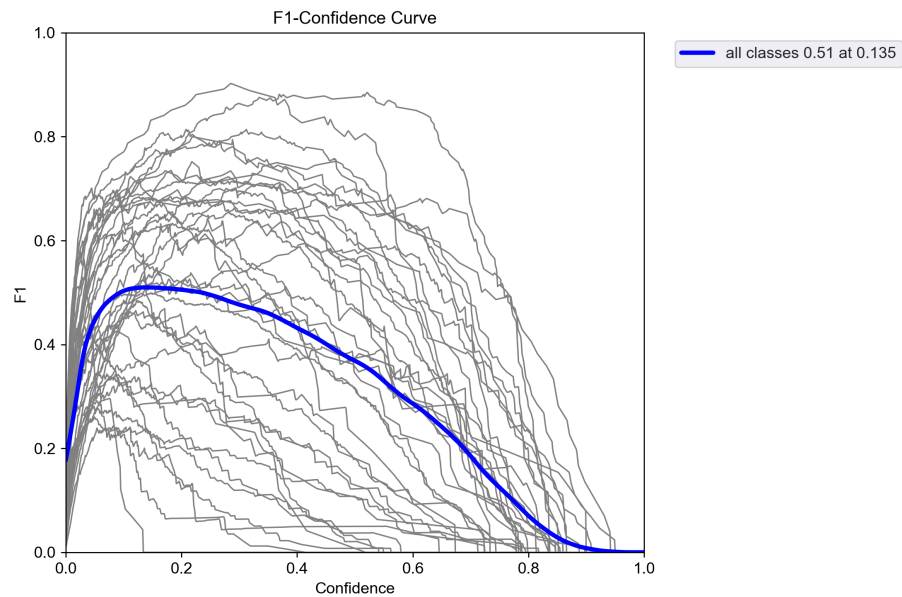
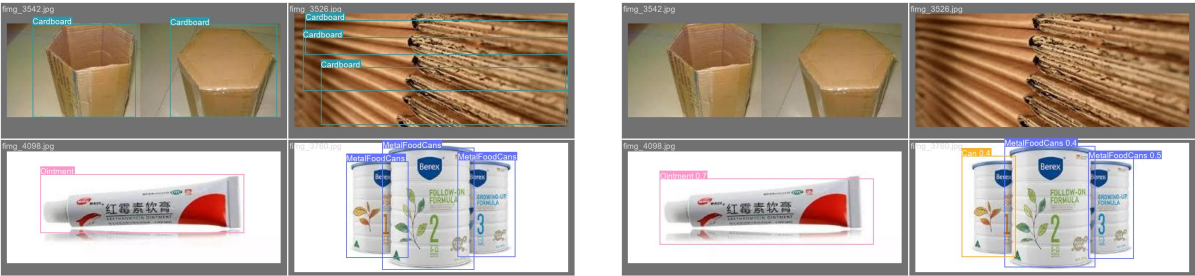


图 6 F1 与置信度阈值关系

4.2 验证集检测效果

图 7 对比了验证集人工标注与模型预测结果。模型能够定位部分纸板、药膏、金属食品罐等目标，但仍存在漏检、类别混淆和低置信度问题。混淆矩阵也显示，不同类别的识别能力差异较大，且部分真实目标被判断为背景。



(a) 验证集真实标注 (b) 模型预测结果

图 7 验证集标注与预测对比

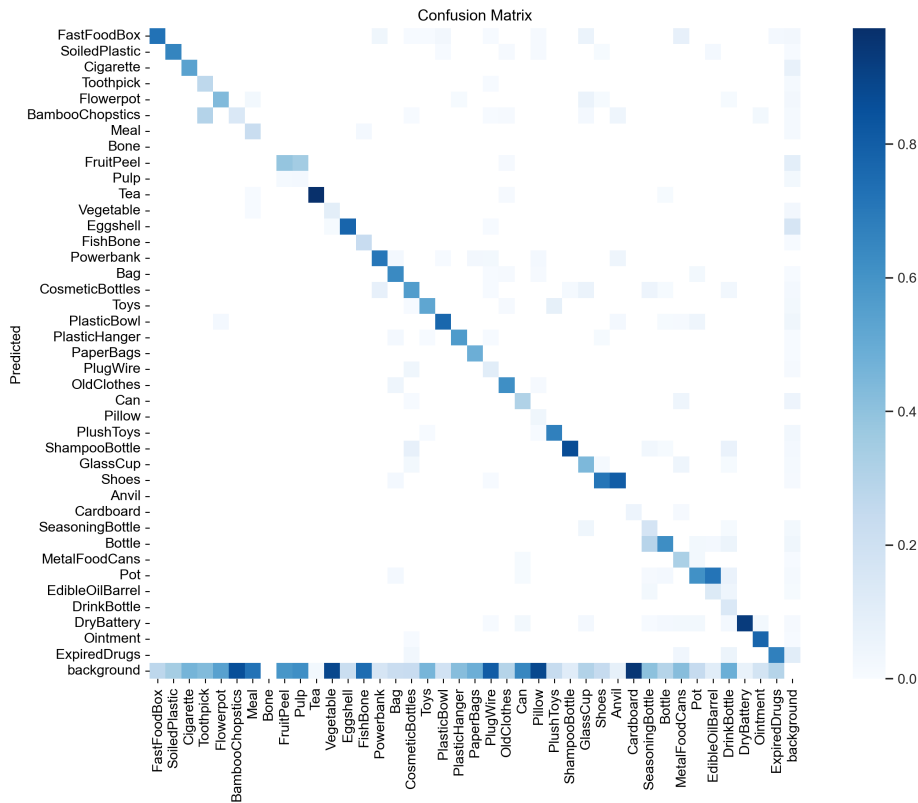


图 8 模型归一化混淆矩阵

### 4.3 有害垃圾检测与前端系统

本项目将干电池、过期药品、药膏、充电宝和化妆品瓶配置为重点告警类别。服务端在启动时加载一次模型，用户上传图片后依次完成预处理、推理、类别统计和风险判断。若检测到有害垃圾，前端会使用红色提示区域展示有害类别及数量；若未检测到有害垃圾，则显示普通检测结果。前端还提供置信度阈值调整功能，便于在高召回和低误报之间进行权衡。

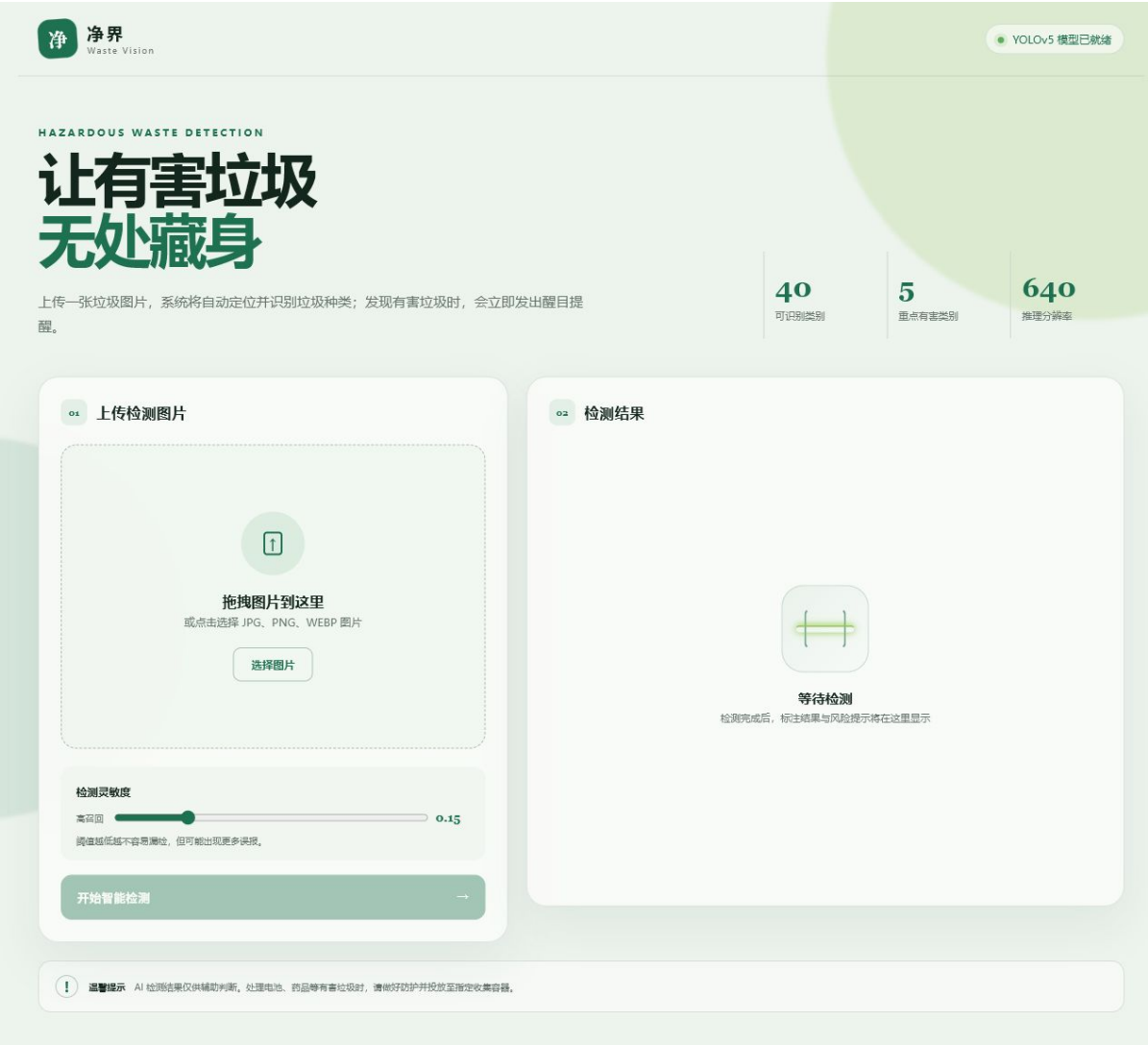


图 9 有害垃圾智能检测 Web 前端

Web 应用支持一键启动，完整 Pipeline 如下：

Listing 5 部署后的实际检测流程

```
Upload image
-> EXIF orientation correction and RGB conversion
-> preprocess.py
-> YOLOv5 best.pt inference
-> class counting and hazardous-waste decision
-> annotated image and highlighted warning
```



## 5 相关讨论

### 5.1 当前系统的有效性

本项目的主要贡献是建立了一条完整、可运行的有害垃圾视觉筛查流程。相较于仅训练模型，本项目还将图像增强、有害类别判断、检测结果可视化和一键启动部署整合为可交互系统。该系统能够识别多类常见垃圾，并对部分常见有害垃圾进行重点提醒，从而初步验证了“拍摄垃圾堆图像并辅助环卫工人寻找有害垃圾”这一思路的可行性。

对于现实应用而言，本系统最有价值的输出并不是单纯给出垃圾名称，而是指出“哪里可能存在有害垃圾”。目标框能够帮助工作人员缩小检查范围，风险提示则能提醒工作人员采用更谨慎的处理方式。即使系统尚不能完全自动化分拣，它仍可作为人工操作前的辅助筛查环节。

### 5.2 局限性分析

1. **训练域与真实垃圾堆存在差异。**现有数据中包含较多背景干净、目标清晰的图片，而真实垃圾堆通常具有严重遮挡、污损和密集堆叠。
2. **整体检测精度仍有限。**当前 Precision 与 Recall 均约为 55%，不能将模型输出视为最终判定结果。
3. **类别粒度与有害属性并不完全一致。**某些容器是否有害取决于其残留物，仅凭外观类别可能无法准确判断。
4. **预处理的视觉改善不等同于检测指标必然提升。**批量对比实验表明，处理后暗部纹理、标签和边缘更清晰，但增强与锐化也可能改变模型训练时学习到的图像分布。因此仍需通过消融实验验证每个组件对 mAP 和 Recall 的真实贡献。
5. **当前缺少专门的垃圾堆独立测试集。**若要证明系统能够稳定用于复杂垃圾堆场景，需要收集并单独标注真实混合垃圾图像进行测试。

### 5.3 YOLO26 与未来模型升级

本实验使用 YOLOv5s 完成基线系统。经调研，Ultralytics 官方文档已发布 YOLO26，并将其描述为面向边缘设备的端到端视觉模型，强调免 NMS 推理和部署效率 [2]。这些特性与本项目未来面向移动终端、现场设备和实时筛查的方向具有较高相关性。

不过，模型版本更新并不意味着在本项目数据集上必然获得更高效果。未来应在相同训练集、验证集、输入尺寸和评价指标下，对 YOLOv5s 与 YOLO26 的 Precision、

Recall、mAP、推理速度和资源占用进行公平对照，再决定是否迁移。特别是本项目更关注有害垃圾漏检，因此模型升级评价应优先观察有害类别 Recall，而非只比较总体 mAP。

## 6 总结与未来工作

本项目针对环卫工人从大量混合垃圾中快速发现有害垃圾的现实需求，完成了从数据获取、图像预处理、YOLOv5s 迁移学习、检测效果分析到 Web 应用部署的完整实验。项目经历了自行拍摄和 AI 生成数据不足的问题，随后采用开源标注数据集扩大训练规模；同时建立独立预处理实验模块，对 50 张课程实验图片和 48 张有害垃圾图片进行批量处理与分类对比，并结合 Resize、Gaussian Blur、CLAHE 和 Unsharp Masking 改善模型输入质量。最终系统能够输出垃圾类别和位置，并在检测到有害垃圾时进行重点提醒。

当前结果证明了方案的可行性，但距离实际稳定应用仍有差距。下一阶段计划包括：

1. 收集真实垃圾堆、转运站和分拣现场图片，建立独立测试集；
2. 增加干电池、药品、化学品容器等有害垃圾样本，尤其补充遮挡、污损、小目标和破损状态；
3. 进行预处理消融实验，量化 Resize、去噪、CLAHE 和锐化的独立贡献；
4. 针对有害垃圾类别调整损失权重与阈值，优先提升 Recall；
5. 在同一数据集上比较 YOLOv5s 与 YOLO26 等模型的准确率、速度与部署成本；
6. 将系统扩展到摄像头实时检测，并探索与自动分拣设备的联动。

总体而言，本项目所解决的关键问题是：在有害垃圾进入下游处理环节之前，利用图像检测帮助工作人员更快地发现并分离风险目标。围绕这一目标继续优化数据、模型和部署方式，能够使系统逐步从课程实验原型发展为具有实际辅助价值的环境保护工具。

7 团队成员贡献

表 4 团队成员贡献说明

| 成员  | 主要贡献                                                                                       |
|-----|--------------------------------------------------------------------------------------------|
| 刘晋璋 | 负责 YOLOv5s 模型训练、训练参数设置、断点续训、权重选择及训练结果整理。                                                   |
| 邓荣基 | 负责图片处理方案设计与实现，包括 Resize、Gaussian Blur、CLAHE、Unsharp Masking、批量处理实验、中文路径兼容及预处理 Pipeline 接入。 |
| 袁致朗 | 负责项目技术调研、现实应用价值分析、实验结果讨论与实验报告撰写。                                                           |

A 主要程序与使用方式

A.1 程序结构

| 文件或目录                                       | 功能                                          |
|---------------------------------------------|---------------------------------------------|
| preprocess.py                               | 图像尺寸调整、去噪、局部对比度增强与锐化。                       |
| preprocess/preprocess.py                    | 面向课程图片与有害垃圾分类目录的批量预处理、中文路径兼容和对比图生成。         |
| preprocess/<br>preprocessing_report.<br>tex | 图像预处理模块的独立实验说明与结果分析。                        |
| train.py                                    | YOLOv5 模型训练与断点续训。                           |
| detect.py                                   | YOLOv5 基础图片检测程序。                            |
| exp12/weights/best.pt                       | 验证集表现最好的模型权重。                               |
| hazardous_waste_web/<br>app.py              | Web 服务、预处理 Pipeline、模型推理与有害垃圾判断。            |
| hazardous_waste_web/<br>static/<br>一键启动.bat | 图片上传、结果展示和风险提示前端。<br>自动检查环境、加载模型并打开 Web 页面。 |

## A.2 运行方式

Listing 6 命令行启动方式

```
conda activate YOLO
python hazardous_waste_web/app.py
```

Windows 环境下也可以直接双击仓库根目录中的一键启动.bat。项目代码仓库地址为: <https://github.com/AndyLongest/HarmfulGarbageDetector.git>。

## 参考文献

- [1] Ultralytics. YOLOv5 GitHub Repository. <https://github.com/ultralytics/yolov5>.
- [2] Ultralytics. YOLO26 Documentation. <https://docs.ultralytics.com/models/yolo26/>.
- [3] ai53\_19. Garbage Datasets. [https://ai.gitcode.com/ai53\\_19/garbage\\_datasets.git](https://ai.gitcode.com/ai53_19/garbage_datasets.git).
- [4] Karel Zuiderveld. Contrast Limited Adaptive Histogram Equalization. In *Graphics Gems IV*, 1994.
- [5] A. Polesel, G. Ramponi, and V. J. Mathews. Image Enhancement via Adaptive Unsharp Masking. *IEEE Transactions on Image Processing*, 9(3):505–510, 2000.